Project Report

RESUME BASED INTERVIEW PREPARATION APP

Submitted by Jiffy Jacob

An AI interview coach that uses retrieval augmented generation to tailor questions and feedback to a candidate's resume

Project detail	Information
Project title	Resume Based AI Interview Preparation Application
Domain	Generative AI, natural language processing, interview preparation
Application	AI Interview Coach
Programme / certification	Data Science with Gen AI, Entri Software Private Limited, Illinois Tech US, NSDC (2026)
Date of report	12 November 2026

Abstract

This project develops a resume-based mock interview coach using Retrieval-Augmented Generation (RAG) and a large language model. A candidate uploads a text-based PDF resume, chooses a target role and an interview round, and practises through typed or spoken answers. The application removes contact details, splits the resume into section-aware chunks, embeds them with all-MiniLM-L6-v2 and retrieves relevant passages from a FAISS index. A Groq-hosted chat model uses these passages to generate personalized Technical or HR questions and to evaluate answers with a ten-point rubric, returning strengths, missing concepts, corrections, an improved answer and a follow-up question. An overall score out of five is computed for each completed round. A notebook-based evaluation rated the system 4.00/5 across three model-judged test cases; this is an early qualitative signal rather than a validated accuracy measure, and context relevance (2.33/5) shows that retrieval needs further work.

Keywords: Retrieval-Augmented Generation, resume parsing, semantic search, FAISS, sentence embeddings, generative AI, mock interview, speech recognition, Streamlit.

1. Introduction

1.1 Background

Interview preparation often requires candidates to find relevant questions, practise aloud, judge their own answers and decide how to improve. Generic question lists may not reflect a candidate's actual projects or skills, while self-assessment can be inconsistent. This project turns a resume into a temporary source of evidence for personalized practice, combining semantic retrieval with generative responses so that questions and coaching refer to the uploaded document and avoid unsupported claims about the candidate's experience.

1.2 Objectives

Extract readable text from an uploaded PDF resume and remove common contact details before it is used in prompts.

Chunk and embed resume content, then retrieve passages relevant to a role, question or answer using FAISS.

Generate Technical and HR interview questions with a Groq-hosted language model.

Accept typed answers and speech input, with an editable transcript in the web application.

Evaluate answers using a defined rubric and return strengths, improvement points, a suggested answer and a follow-up question.

Calculate a round-level score out of five and provide a Streamlit interface suitable for a practice session.

Assess initial system quality with a small model-as-judge test set and identify retrieval weaknesses.

1.3 Scope

The system is a practice and coaching tool. It does not authenticate users, compare a resume against a job description, store interview history, certify competence or make hiring decisions. It supports PDF text extraction, role selection, Technical and HR rounds, and typed or microphone-based answers within the active Streamlit session.

2. User workflow

The user uploads a PDF, reviews the cleaned text, selects a target role and chooses a Technical or HR round. The app generates a question; the user answers by typing or by speaking and editing the transcript, then submits. The system retrieves resume evidence related to the question and answer, requests a structured evaluation and displays coaching feedback. The interview continues with new questions and adaptive follow-ups. On exit, evaluated answers are averaged and converted from a ten-point scale to a five-point overall score. The app also rejects resumes with fewer than 100 characters of extractable text and keeps the session state in Streamlit.

3. System design and architecture

3.1 Processing pipeline

Stage	Input and operation	Output
1. Upload and parse	PDF bytes are read with pypdf; text from each page is concatenated.	Raw resume text
2. Clean	Unicode and whitespace are normalized; email, phone and URL patterns are removed.	Reduced-identifiability resume text
3. Chunk	Text is separated using detected section headings and packed into chunks of about 450 characters, with limited overlap at chunk boundaries.	Labeled resume passages
4. Embed and index	all-MiniLM-L6-v2 produces normalized vectors; FAISS IndexFlatIP stores the vectors.	In-memory semantic index
5. Retrieve	The query is embedded and the most similar passages are selected by inner product, equivalent to cosine similarity for normalized vectors.	Relevant resume evidence
6. Generate and evaluate	Prompts combine role, interview type, question or answer, rubric and retrieved evidence; the Groq API returns structured content.	Questions, feedback and follow-up
7. Present	Streamlit renders the interview flow, editable answer, coaching feedback and round score.	Interactive practice session

3.2 Application components

The supplied app.py contains the complete Streamlit application: resume cleaning and chunking, the cached sentence-transformer loader, FAISS indexing and retrieval, prompt construction, Groq API calls, response validation, session-state interview logic, speech transcription and the user interface. The notebook records the development and evaluation workflow, using approximately 500-character chunks and top-three retrieval; the app refines this into section-aware chunks of about 450 characters with role- and round-aware retrieval queries.

3.3 Retrieval-Augmented Generation design

RAG separates retrieval from generation. First, the system embeds the candidate’s resume chunks and finds passages similar to a query. Then it includes selected evidence in the model prompt. For Technical questions, the app retrieves content using the role, chosen focus areas and resume project terms; it can add a project spotlight chunk. For HR questions, it retrieves content related to behavioural themes and candidate background, but instructs the model not to ask technical questions. During answer evaluation, the query includes the interview question and a portion of the candidate’s answer. The app then requests evidence-aware evaluation while permitting general technical knowledge for conceptual questions.

4. Implementation details

4.1 Resume extraction and privacy filtering

The app extracts PDF text with pypdf, normalizes Unicode and whitespace, removes control characters, replaces unusual bullet symbols, and strips email, URL and phone patterns before the text is indexed or added to prompts.

4.2 Embeddings and semantic search

The embedding model is sentence-transformers/all-MiniLM-L6-v2, cached as a Streamlit resource so it is not reloaded on every rerun. Passages are encoded with normalized embeddings and stored in an in-memory FAISS IndexFlatIP index built for the active resume rather than saved as a permanent corpus.

4.3 Generative model and structured output

The notebook uses openai/gpt-oss-20b through the Groq API. The Streamlit app checks Groq's available model list and prefers a configured sequence (Llama 3.1 8B, Llama 3.3 70B, GPT-OSS), with an optional override through secrets, so the active model can change over time. Evaluation prompts request JSON; the app parses the response, clamps each score to its rubric maximum, normalizes feedback fields and does not count an unevaluated answer toward the average.

4.4 Evaluation rubric

Dimension	Maximum	Interpretation in app.py
Technical correctness	4	Factual accuracy and correctness of the answer, with the rubric adjusted to the interview round.
Conceptual understanding	3	Evidence of understanding; for HR answers, depth and self-awareness.
Completeness	2	Coverage of essential points; for HR, a concrete situation, action and result where appropriate.
Clarity	1	Whether the answer is understandable and clearly communicated.
Total per answer	10	The application sums validated dimensions and computes the round average.

Technical and HR answers use different prompt guidance. The model is asked not to penalize simple but correct explanations, not to double-count omissions, not to invent personal facts in suggested HR answers, and to give exactly one follow-up question.

4.5 Voice input

The notebook prototype records from a local microphone and transcribes with faster-whisper on the local machine. The deployed Streamlit app instead uses the browser audio input component and sends the WAV audio to the Groq transcription endpoint, preferring Whisper large-v3-turbo with Whisper large-v3 as fallback. The transcript appears in an editable text area before evaluation.

5. Evaluation and results

5.1 Evaluation method

The notebook includes three test cases: a general Random Forest question, a resume-project question about a book recommendation project and an HR question about disagreement with a teammate. For each case, the notebook evaluates the answer and asks a language model acting as judge to rate context relevance, answer grounding, evaluation usefulness and feedback relevance from one to five. The notebook explicitly describes this as an automated model-based assessment rather than a verified ground-truth accuracy score.

5.2 Observed results and validity

Judge dimension	Mean score
Context relevance	2.33 / 5
Answer grounding	4.67 / 5
Evaluation usefulness	4.33 / 5
Feedback relevance	4.67 / 5
Overall model-quality rating	4.00 / 5

Answer grounding and feedback relevance were rated highly, but context relevance was weak: the passage retrieved for the general Random Forest question was unrelated, while the project-specific test retrieved relevant content. Retrieval is the clearest improvement area.

The evaluation has only three examples and uses the same model provider for generating and judging outputs. It includes no human ratings, labeled retrieval precision, statistical uncertainty or variety of resumes and roles. The 4.00/5 must therefore be read as a small model-judged quality rating, not as 80 percent accuracy, and it motivates a larger, manually reviewed test set.

6. Security, privacy and responsible use

The code removes common contact identifiers before resume text is used in prompts, and retains resume text, chunks, the FAISS index, questions and answers only in the current Streamlit session; there is no database or persistent history. However, retrieved resume passages and candidate answers are sent to Groq for question generation and evaluation, and spoken audio is sent to Groq for transcription. This description is limited to the supplied code and does not state the third-party provider's retention practices. Users should be told clearly about these transfers and avoid uploading information they are unwilling to process through those services. Contact-detail removal reduces exposure but does not anonymize a resume, since names, employers, education and projects may remain. Scores and feedback are coaching suggestions, not an objective measure of employability.

7. Limitations

PDF text extraction depends on embedded text; scanned or complex layouts can produce empty or poorly ordered text.

Question diversity and answer evaluation depend on an external language model and may vary with model availability, prompts and API limits.

The application does not persist interview history, provide a job-description comparison, offer multi-user accounts or support multilingual evaluation.

Prompt instructions reduce unsupported details but cannot guarantee that every generated question, score or suggested answer is correct.

Tools, libraries and project links

Area	Technology or reference
Language and UI	Python; Streamlit
PDF processing	pypdf
Embeddings	Sentence Transformers; all-MiniLM-L6-v2
Vector search	FAISS (IndexFlatIP with normalized vectors)
Generative AI and transcription	Groq API; dynamically selected chat model; Groq Whisper in app.py
Notebook speech prototype	faster-whisper, sounddevice and SciPy for local recording/transcription
Project source	https://github.com/jiffy-jacob/Resume-based-Interview-Preparation-application
Live application	https://resume-based-interview-preparation-application-j59wvvzlu7dwmqu.streamlit.app/
Author profile	https://github.com/jiffy-jacob

Conclusion

The AI Interview Coach brings resume parsing, semantic retrieval and generative AI together in an interactive interview practice workflow. It supports Technical and HR sessions, typed and spoken answers, structured feedback and an overall score. The initial three-case evaluation rated the system 4.00 out of 5, while identifying context relevance as the main weakness. The project provides a useful foundation for personalized practice; broader human-reviewed evaluation and stronger retrieval are needed before drawing conclusions about its reliability.

Future Scope

Improve chunking and query construction, and benchmark retrieval against labeled resume passages using precision and recall at different k values.

Expand testing to more resumes, job roles, technical and HR questions, and answer quality levels; compare model feedback with human reviewers.

Add automated checks for PDF extraction, personal-detail removal, retrieval relevance, scoring limits, malformed model responses and API failures.

Offer local speech transcription as an option and make the data flow and third-party processing clear to users.

Add accessibility improvements, better support for scanned or complex-layout PDFs, and clearer controls for ending and resetting a session.